# Chapter 9. Subqueries

Subqueries are a powerful tool that you can use in all four SQL data statements. In this chapter, I'll show you how subqueries can be used to filter data, generate values, and construct temporary data sets. After a little experimentation, I think you'll agree that subqueries are one of the most powerful features of the SQL language.

## What Is a Subquery?

A *subquery* is a query contained within another SQL statement (which I refer to as the *containing statement* for the rest of this discussion). A subquery is always enclosed within parentheses, and it is usually executed prior to the containing statement. Like any query, a subquery returns a result set that may consist of:

- A single row with a single column
- Multiple rows with a single column
- Multiple rows having multiple columns

The type of result set returned by the subquery determines how it may be used and which operators the containing statement may use to interact with the data the subquery returns. When the containing statement has finished executing, the data returned by any subqueries is discarded, making a subquery act like a temporary table with *statement scope* (meaning that the server frees up any memory allocated to the subquery results after the SQL statement has finished execution).

You already saw several examples of subqueries in earlier chapters, but here's a simple example to get started:

```
mysql> SELECT customer_id, first_name, last_name
    -> FROM customer
    -> WHERE customer_id = (SELECT MAX(customer_id) FROM customer);
+-------------+------------+-----------+
| customer_id | first_name | last_name |
+-------------+------------+-----------+
|         599 | AUSTIN     | CINTRON   |
```

```
+-------------+------------+-----------+
1 row in set (0.27 sec)
```

In this example, the subquery returns the maximum value found in the `customer_id` column in the `customer` table, and the containing statement then returns data about that customer. If you are ever confused about what a subquery is doing, you can run the subquery by itself (without the parentheses) to see what it returns. Here's the subquery from the previous example:

```
mysql> SELECT MAX(customer_id) FROM customer;
+------------------+
| MAX(customer_id) |
+------------------+
|              599 |
+------------------+
1 row in set (0.00 sec)
```

The subquery returns a single row with a single column, which allows it to be used as one of the expressions in an equality condition (if the subquery returned two or more rows, it could be *compared* to something but could not be *equal* to anything, but more on this later). In this case, you can take the value the subquery returned and substitute it into the righthand expression of the filter condition in the containing query, as in the following:

```
mysql> SELECT customer_id, first_name, last_name
    -> FROM customer
    -> WHERE customer_id = 599;
+-------------+------------+-----------+
| customer_id | first_name | last_name |
+-------------+------------+-----------+
|         599 | AUSTIN     | CINTRON   |
+-------------+------------+-----------+
1 row in set (0.00 sec)
```

The subquery is useful in this case because it allows you to retrieve information about the customer with the highest ID in a single query, rather than retrieving the maximum `customer_id` using one query and then writing a second query to retrieve the desired data from the `customer` table. As you will see, subqueries are useful in many other situations as well and may become one of the most powerful tools in your SQL toolkit.

# Subquery Types

Along with the differences noted previously regarding the type of result set returned by a subquery (single row/column, single row/multicolumn, or multiple columns), you can use another feature to differentiate subqueries; some subqueries are completely self-contained (called *noncorrelated subqueries*), while others reference columns from the containing statement (called *correlated subqueries*). The next several sections explore these two subquery types and show the different operators that you can employ to interact with them.

# Noncorrelated Subqueries

The example from earlier in the chapter is a noncorrelated subquery; it may be executed alone and does not reference anything from the containing statement. Most subqueries that you encounter will be of this type unless you are writing `update` or `delete` statements, which frequently make use of correlated subqueries (more on this later). Along with being noncorrelated, the example from earlier in the chapter also returns a result set containing a single row and column. This type of subquery is known as a *scalar subquery* and can appear on either side of a condition using the usual operators ( `=` , `<>` , `<` , `>` , `<=` , `>=` ). The next example shows how you can use a scalar subquery in an inequality condition:

```
mysql> SELECT city_id, city
    -> FROM city
    -> WHERE country_id <>
    ->  (SELECT country_id FROM country WHERE country = 'India');
+---------+----------------------------+
| city_id | city                       |
+---------+----------------------------+
|       1 | A Corua (La Corua)         |
|       2 | Abha                       |
|       3 | Abu Dhabi                  |
|       4 | Acua                       |
|       5 | Adana                      |
|       6 | Addis Abeba                |
...
|     595 | Zapopan                    |
|     596 | Zaria                      |
|     597 | Zeleznogorsk               |
|     598 | Zhezqazghan                |
```

```
|    599 | Zhoushan                  |
|    600 | Ziguinchor                |
+---------+---------------------------+
540 rows in set (0.02 sec)
```

This query returns all cities that are not in India. The subquery, which is found on the last line of the statement, returns the country ID for India, and the containing query returns all cities that do not have that country ID. While the subquery in this example is quite simple, subqueries may be as complex as you need them to be, and they may utilize any and all the available query clauses ( `select` , `from` , `where` , `group by` , `having` , and `order by` ).

If you use a subquery in an equality condition but the subquery returns more than one row, you will receive an error. For example, if you modify the previous query such that the subquery returns *all* countries *except for* India, you will receive the following error:

```
mysql> SELECT city_id, city
    -> FROM city
    -> WHERE country_id <>
    ->  (SELECT country_id FROM country WHERE country <> 'India');
ERROR 1242 (21000): Subquery returns more than 1 row
```

If you run the subquery by itself, you will see the following results:

```
mysql> SELECT country_id FROM country WHERE country <> 'India';
+------------+
| country_id |
+------------+
|          1 |
|          2 |
|          3 |
|          4 |
...
|        106 |
|        107 |
|        108 |
|        109 |
+------------+
108 rows in set (0.00 sec)
```

The containing query fails because an expression ( `country_id` ) cannot be equated to a set of expressions ( `country_id` s 1, 2, 3, ..., 109). In other words, a single thing cannot be equated to a set of things. In the next section, you will see how to fix the problem by using a different operator.

## Multiple-Row, Single-Column Subqueries

If your subquery returns more than one row, you will not be able to use it on one side of an equality condition, as the previous example demonstrated. However, there are four additional operators that you can use to build conditions with these types of subqueries.

### The in and not in operators

While you can't *equate* a single value to a set of values, you can check to see whether a single value can be found *within* a set of values. The next example, while it doesn't use a subquery, demonstrates how to build a condition that uses the `in` operator to search for a value within a set of values:

```
mysql> SELECT country_id
    -> FROM country
    -> WHERE country IN ('Canada','Mexico');
+------------+
| country_id |
+------------+
|         20 |
|         60 |
+------------+
2 rows in set (0.00 sec)
```

The expression on the lefthand side of the condition is the `country` column, while the righthand side of the condition is a set of strings. The `in` operator checks to see whether either of the strings can be found in the `country` column; if so, the condition is met, and the row is added to the result set. You could achieve the same results using two equality conditions, as in:

```
mysql> SELECT country_id
    -> FROM country
    -> WHERE country = 'Canada' OR country = 'Mexico';
+------------+
| country_id |
+------------+
```

```
|       20 |
|       60 |
+------------+
2 rows in set (0.00 sec)
```

While this approach seems reasonable when the set contains only two expressions, it is easy to see why a single condition using the `in` operator would be preferable if the set contained dozens (or hundreds, thousands, etc.) of values.

Although you will occasionally create a set of strings, dates, or numbers to use on one side of a condition, you are more likely to generate the set using a subquery that returns one or more rows. The following query uses the `in` operator with a subquery on the righthand side of the filter condition to return all cities that are in Canada or Mexico:

```
mysql> SELECT city_id, city
    -> FROM city
    -> WHERE country_id IN
    ->  (SELECT country_id
    ->   FROM country
    ->   WHERE country IN ('Canada','Mexico'));
+---------+----------------------------+
| city_id | city                       |
+---------+----------------------------+
|     179 | Gatineau                   |
|     196 | Halifax                    |
|     300 | Lethbridge                 |
|     313 | London                     |
|     383 | Oshawa                     |
|     430 | Richmond Hill              |
|     565 | Vancouver                  |
...
|     452 | San Juan Bautista Tuxtepec |
|     541 | Torren                     |
|     556 | Uruapan                    |
|     563 | Valle de Santiago          |
|     595 | Zapopan                    |
+---------+----------------------------+
37 rows in set (0.00 sec)
```

Along with seeing whether a value exists within a set of values, you can check the converse using the `not in` operator. Here's another version of the previous query using `not in` instead of `in`:

```
mysql> SELECT city_id, city
    -> FROM city
    -> WHERE country_id NOT IN
    ->  (SELECT country_id
    ->   FROM country
    ->   WHERE country IN ('Canada','Mexico'));
+---------+----------------------------+
| city_id | city                       |
+---------+----------------------------+
|       1 | A Corua (La Corua)         |
|       2 | Abha                       |
|       3 | Abu Dhabi                  |
|       5 | Adana                      |
|       6 | Addis Abeba                |
...
|     596 | Zaria                      |
|     597 | Zeleznogorsk               |
|     598 | Zhezqazghan                |
|     599 | Zhoushan                   |
|     600 | Ziguinchor                 |
+---------+----------------------------+
563 rows in set (0.00 sec)
```

This query finds all cities that are *not* in Canada or Mexico.

### The all operator

While the `in` operator is used to see whether an expression can be found within a set of expressions, the `all` operator allows you to make comparisons between a single value and every value in a set. To build such a condition, you will need to use one of the comparison operators ( `=` , `<>` , `<` , `>` , etc.) in conjunction with the `all` operator. For example, the next query finds all customers who have never gotten a free film rental:

```
mysql> SELECT first_name, last_name
    -> FROM customer
    -> WHERE customer_id <> ALL
    ->  (SELECT customer_id
    ->   FROM payment
    ->   WHERE amount = 0);
+------------+-------------+
| first_name | last_name   |
+------------+-------------+
| MARY       | SMITH       |
```

```
| PATRICIA    | JOHNSON      |
| LINDA       | WILLIAMS     |
| BARBARA     | JONES        |
...
| EDUARDO     | HIATT        |
| TERRENCE    | GUNDERSON    |
| ENRIQUE     | FORSYTHE     |
| FREDDIE     | DUGGAN       |
| WADE        | DELVALLE     |
| AUSTIN      | CINTRON      |
+-------------+--------------+
576 rows in set (0.01 sec)
```

The subquery returns the set of IDs for customers who have paid $0 for a film rental, and the containing query returns the names of all customers whose ID is not in the set returned by the subquery. If this approach seems a bit clumsy to you, you are in good company; most people would prefer to phrase the query differently and avoid using the `all` operator. To illustrate, the previous query generates the same results as the next example, which uses the `not in` operator:

```
SELECT first_name, last_name
FROM customer
WHERE customer_id NOT IN
 (SELECT customer_id
  FROM payment
  WHERE amount = 0)
```

It's a matter of preference, but I think that most people would find the version that uses `not in` to be easier to understand.

---

NOTE

When using `not in` or `<> all` to compare a value to a set of values, you must be careful to ensure that the set of values does not contain a `null` value, because the server equates the value on the lefthand side of the expression to each member of the set, and any attempt to equate a value to `null` yields `unknown`. Thus, the following query returns an empty set:

```
mysql> SELECT first_name, last_name
    -> FROM customer
    -> WHERE customer_id NOT IN (122, 452, NULL);
Empty set (0.00 sec)
```

---

Here's another example using the `all` operator, but this time the subquery is in the `having` clause:

```
mysql> SELECT customer_id, count(*)
    -> FROM rental
    -> GROUP BY customer_id
    -> HAVING count(*) > ALL
    ->  (SELECT count(*)
    ->   FROM rental r
    ->    INNER JOIN customer c
    ->    ON r.customer_id = c.customer_id
    ->    INNER JOIN address a
    ->    ON c.address_id = a.address_id
    ->    INNER JOIN city ct
    ->    ON a.city_id = ct.city_id
    ->    INNER JOIN country co
    ->    ON ct.country_id = co.country_id
    ->   WHERE co.country IN ('United States','Mexico','Canada')
    ->   GROUP BY r.customer_id
    ->   );
+-------------+----------+
| customer_id | count(*) |
+-------------+----------+
|       148 |      46 |
+-------------+----------+
1 row in set (0.01 sec)
```

The subquery in this example returns the total number of film rentals for all customers in North America, and the containing query returns all customers whose total number of film rentals exceeds any of the North American customers.

### The any operator

Like the `all` operator, the `any` operator allows a value to be compared to the members of a set of values; unlike `all`, however, a condition using the `any` operator evaluates to `true` as soon as a single comparison is favorable. This example will find all customers whose total film rental payments exceed the total payments for all customers in Bolivia, Paraguay, or Chile:

```
mysql> SELECT customer_id, sum(amount)
    -> FROM payment
    -> GROUP BY customer_id
```

```
    -> HAVING sum(amount) > ANY
    ->  (SELECT sum(p.amount)
    ->   FROM payment p
    ->    INNER JOIN customer c
    ->    ON p.customer_id = c.customer_id
    ->    INNER JOIN address a
    ->    ON c.address_id = a.address_id
    ->    INNER JOIN city ct
    ->    ON a.city_id = ct.city_id
    ->    INNER JOIN country co
    ->    ON ct.country_id = co.country_id
    ->   WHERE co.country IN ('Bolivia','Paraguay','Chile')
    ->   GROUP BY co.country
    ->  );
+-------------+-------------+
| customer_id | sum(amount) |
+-------------+-------------+
|         137 |      194.61 |
|         144 |      195.58 |
|         148 |      216.54 |
|         178 |      194.61 |
|         459 |      186.62 |
|         526 |      221.55 |
+-------------+-------------+
6 rows in set (0.03 sec)
```

The subquery returns the total film rental fees for all customers in Bolivia, Paraguay, and Chile, and the containing query returns all customers who outspent at least one of these three countries (if you find yourself outspending an entire country, perhaps you need to cancel your Netflix subscription and book a trip to Bolivia, Paraguay, or Chile...).

---

NOTE

Although most people prefer to use `in`, using `= any` is equivalent to using the `in` operator.

---

## Multicolumn Subqueries

So far, the subquery examples in this chapter have returned a single column and one or more rows. In certain situations, however, you can use subqueries that return two or more columns. To show the utility of multicolumn subqueries, it might help to look first at an example that uses multiple, single-column subqueries:

```
mysql> SELECT fa.actor_id, fa.film_id
    -> FROM film_actor fa
    -> WHERE fa.actor_id IN
    ->  (SELECT actor_id FROM actor WHERE last_name = 'MONROE')
    ->   AND fa.film_id IN
    ->  (SELECT film_id FROM film WHERE rating = 'PG');
+----------+---------+
| actor_id | film_id |
+----------+---------+
|      120 |      63 |
|      120 |     144 |
|      120 |     414 |
|      120 |     590 |
|      120 |     715 |
|      120 |     894 |
|      178 |     164 |
|      178 |     194 |
|      178 |     273 |
|      178 |     311 |
|      178 |     983 |
+----------+---------+
11 rows in set (0.00 sec)
```

This query uses two subqueries to identify all actors with the last name Monroe and all films rated PG, and the containing query then uses this information to retrieve all cases where an actor named Monroe appeared in a PG film. However, you could merge the two single-column subqueries into one multicolumn subquery and compare the results to two columns in the `film_actor` table. To do so, your filter condition must name both columns from the `film_actor` table surrounded by parentheses and in the same order as returned by the subquery, as in:

```
mysql> SELECT actor_id, film_id
    -> FROM film_actor
    -> WHERE (actor_id, film_id) IN
    ->  (SELECT a.actor_id, f.film_id
    ->   FROM actor a
    ->     CROSS JOIN film f
    ->   WHERE a.last_name = 'MONROE'
    ->   AND f.rating = 'PG');
+----------+---------+
| actor_id | film_id |
+----------+---------+
|      120 |      63 |
```

```
|     120 |     144 |
|     120 |     414 |
|     120 |     590 |
|     120 |     715 |
|     120 |     894 |
|     178 |     164 |
|     178 |     194 |
|     178 |     273 |
|     178 |     311 |
|     178 |     983 |
+----------+---------+
11 rows in set (0.00 sec)
```

This version of the query performs the same function as the previous example, but with a single subquery that returns two columns instead of two subqueries that each return a single column. The subquery in this version uses a type of join called a *cross join*, which will be explored in the next chapter. The basic idea is to return all combinations of actors named Monroe (2) and all films rated PG (194) for a total of 388 rows, 11 of which can be found in the `film_actor` table.

# Correlated Subqueries

All of the subqueries shown thus far have been independent of their containing statements, meaning that you can execute them by themselves and inspect the results. A *correlated subquery*, on the other hand, is *dependent* on its containing statement from which it references one or more columns. Unlike a noncorrelated subquery, a correlated subquery is not executed once prior to execution of the containing statement; instead, the correlated subquery is executed once for each candidate row (rows that might be included in the final results). For example, the following query uses a correlated subquery to count the number of film rentals for each customer, and the containing query then retrieves those customers who have rented exactly 20 films:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE 20 =
    ->  (SELECT count(*) FROM rental r
    ->   WHERE r.customer_id = c.customer_id);
+------------+-------------+
| first_name | last_name   |
+------------+-------------+
| LAUREN     | HUDSON      |
```

```
| JEANETTE   | GREENE     |
| TARA       | RYAN       |
| WILMA      | RICHARDS   |
| JO         | FOWLER     |
| KAY        | CALDWELL   |
| DANIEL     | CABRAL     |
| ANTHONY    | SCHWAB     |
| TERRY      | GRISSOM    |
| LUIS       | YANEZ      |
| HERBERT    | KRUGER     |
| OSCAR      | AQUINO     |
| RAUL       | FORTIER    |
| NELSON     | CHRISTENSON |
| ALFREDO    | MCADAMS    |
+------------+-------------+
15 rows in set (0.01 sec)
```

The reference to `c.customer_id` at the very end of the subquery is what makes the subquery correlated; the containing query must supply values for `c.customer_id` for the subquery to execute. In this case, the containing query retrieves all 599 rows from the `customer` table and executes the subquery once for each customer, passing in the appropriate customer ID for each execution. If the subquery returns the value `20`, then the filter condition is met, and the row is added to the result set.

---

**NOTE**

One word of caution: since the correlated subquery will be executed once for each row of the containing query, the use of correlated subqueries can cause performance issues if the containing query returns a large number of rows.

---

Along with equality conditions, you can use correlated subqueries in other types of conditions, such as the range condition illustrated here:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE
    ->  (SELECT sum(p.amount) FROM payment p
    ->   WHERE p.customer_id = c.customer_id)
    ->   BETWEEN 180 AND 240;
+------------+-----------+
| first_name | last_name |
+------------+-----------+
```

```
| RHONDA     | KENNEDY   |
| CLARA      | SHAW      |
| ELEANOR    | HUNT      |
| MARION     | SNYDER    |
| TOMMY      | COLLAZO   |
| KARL       | SEAL      |
+------------+-----------+
6 rows in set (0.03 sec)
```

This variation on the previous query finds all customers whose total payments for all film rentals are between $180 and $240. Once again, the correlated subquery is executed 599 times (once for each customer row), and each execution of the subquery returns the total account balance for the given customer.

---

NOTE

Another subtle difference in the previous query is that the subquery is on the lefthand side of the condition, which may look a bit odd but is perfectly valid.

---

## The exists Operator

While you will often see correlated subqueries used in equality and range conditions, the most common operator used to build conditions that utilize correlated subqueries is the `exists` operator. You use the `exists` operator when you want to identify that a relationship exists without regard for the quantity; for example, the following query finds all the customers who rented at least one film prior to May 25, 2005, without regard for how many films were rented:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE EXISTS
    ->  (SELECT 1 FROM rental r
    ->   WHERE r.customer_id = c.customer_id
    ->     AND date(r.rental_date) < '2005-05-25');
+------------+-------------+
| first_name | last_name   |
+------------+-------------+
| CHARLOTTE  | HUNTER      |
| DELORES    | HANSEN      |
| MINNIE     | ROMERO      |
| CASSANDRA  | WALTERS     |
```

```
| ANDREW     | PURDY      |
| MANUEL     | MURRELL    |
| TOMMY      | COLLAZO    |
| NELSON     | CHRISTENSON |
+------------+-------------+
8 rows in set (0.03 sec)
```

Using the `exists` operator, your subquery can return zero, one, or many rows, and the condition simply checks whether the subquery returned one or more rows. If you look at the `select` clause of the subquery, you will see that it consists of a single literal ( `1` ); since the condition in the containing query only needs to know how many rows have been returned, the actual data the subquery returned is irrelevant. Your subquery can return whatever strikes your fancy, as demonstrated next:

```
mysql> SELECT c.first_name, c.last_name
    -> FROM customer c
    -> WHERE EXISTS
    ->  (SELECT r.rental_date, r.customer_id, 'ABCD' str, 2 * 3 / 7 nmbr
    ->   FROM rental r
    ->   WHERE r.customer_id = c.customer_id
    ->     AND date(r.rental_date) < '2005-05-25');
+------------+-------------+
| first_name | last_name   |
+------------+-------------+
| CHARLOTTE  | HUNTER      |
| DELORES    | HANSEN      |
| MINNIE     | ROMERO      |
| CASSANDRA  | WALTERS     |
| ANDREW     | PURDY       |
| MANUEL     | MURRELL     |
| TOMMY      | COLLAZO     |
| NELSON     | CHRISTENSON |
+------------+-------------+
8 rows in set (0.03 sec)
```

However, the convention is to specify either `select 1` or `select *` when using `exists` .

You may also use `not exists` to check for subqueries that return no rows, as demonstrated by the following:

```
mysql> SELECT a.first_name, a.last_name
    -> FROM actor a
```

```
    -> WHERE NOT EXISTS
    ->  (SELECT 1
    ->   FROM film_actor fa
    ->    INNER JOIN film f ON f.film_id = fa.film_id
    ->   WHERE fa.actor_id = a.actor_id
    ->    AND f.rating = 'R');
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| JANE       | JACKMAN   |
+------------+-----------+
1 row in set (0.00 sec)
```

This query finds all actors who have never appeared in an R-rated film.

## Data Manipulation Using Correlated Subqueries

All of the examples thus far in the chapter have been `select` statements, but don't think that means that subqueries aren't useful in other SQL statements. Subqueries are used heavily in `update`, `delete`, and `insert` statements as well, with correlated subqueries appearing frequently in `update` and `delete` statements. Here's an example of a correlated subquery used to modify the `last_update` column in the `customer` table:

```
UPDATE customer c
SET c.last_update =
 (SELECT max(r.rental_date) FROM rental r
  WHERE r.customer_id = c.customer_id);
```

This statement modifies every row in the `customer` table (since there is no `where` clause) by finding the latest rental date for each customer in the `rental` table. While it seems reasonable to expect that every customer will have at least one film rental, it would be best to check before attempting to update the `last_update` column; otherwise, the column will be set to `null`, since the subquery would return no rows. Here's another version of the `update` statement, this time employing a `where` clause with a second correlated subquery:

```
UPDATE customer c
SET c.last_update =
 (SELECT max(r.rental_date) FROM rental r
  WHERE r.customer_id = c.customer_id)
WHERE EXISTS
```

```
 (SELECT 1 FROM rental r
  WHERE r.customer_id = c.customer_id);
```

The two correlated subqueries are identical except for the `select` clauses. The subquery in the `set` clause, however, executes only if the condition in the `update` statement's `where` clause evaluates to `true` (meaning that at least one rental was found for the customer), thus protecting the data in the `last_update` column from being overwritten with a `null`.

Correlated subqueries are also common in `delete` statements. For example, you may run a data maintenance script at the end of each month that removes unnecessary data. The script might include the following statement, which removes rows from the `customer` table where there have been no film rentals in the past year:

```
DELETE FROM customer
WHERE 365 < ALL
 (SELECT datediff(now(), r.rental_date) days_since_last_rental
  FROM rental r
  WHERE r.customer_id = customer.customer_id);
```

When using correlated subqueries with `delete` statements in MySQL, keep in mind that, for whatever reason, table aliases are not allowed when using `delete`, which is why I had to use the entire table name in the subquery. With most other database servers, you could provide an alias for the `customer` table, such as:

```
DELETE FROM customer c
WHERE 365 < ALL
 (SELECT datediff(now(), r.rental_date) days_since_last_rental
  FROM rental r
  WHERE r.customer_id = c.customer_id);
```

# When to Use Subqueries

Now that you have learned about the different types of subqueries and the different operators that you can employ to interact with the data returned by subqueries, it's time to explore the many ways in which you can use subqueries to build powerful SQL statements. The next three sections demonstrate how you may use subqueries to construct custom tables, to build conditions, and to generate column values in result sets.

## Subqueries as Data Sources

Back in Chapter 3, I stated that the `from` clause of a `select` statement contains the *tables* to be used by the query. Since a subquery generates a result set containing rows and columns of data, it is perfectly valid to include subqueries in your `from` clause along with tables. Although it might, at first glance, seem like an interesting feature without much practical merit, using subqueries alongside tables is one of the most powerful tools available when writing queries. Here's a simple example:

```
mysql> SELECT c.first_name, c.last_name,
    ->   pymnt.num_rentals, pymnt.tot_payments
    -> FROM customer c
    ->   INNER JOIN
    ->    (SELECT customer_id,
    ->       count(*) num_rentals, sum(amount) tot_payments
    ->     FROM payment
    ->     GROUP BY customer_id
    ->    ) pymnt
    ->   ON c.customer_id = pymnt.customer_id;
+------------+-------------+-------------+--------------+
| first_name | last_name   | num_rentals | tot_payments |
+------------+-------------+-------------+--------------+
| MARY       | SMITH       |          32 |       118.68 |
| PATRICIA   | JOHNSON     |          27 |       128.73 |
| LINDA      | WILLIAMS    |          26 |       135.74 |
| BARBARA    | JONES       |          22 |        81.78 |
| ELIZABETH  | BROWN       |          38 |       144.62 |
...
| TERRENCE   | GUNDERSON   |          30 |       117.70 |
| ENRIQUE    | FORSYTHE    |          28 |        96.72 |
| FREDDIE    | DUGGAN      |          25 |        99.75 |
| WADE       | DELVALLE    |          22 |        83.78 |
| AUSTIN     | CINTRON     |          19 |        83.81 |
+------------+-------------+-------------+--------------+
599 rows in set (0.03 sec)
```

In this example, a subquery generates a list of customer IDs along with the number of film rentals and the total payments. Here's the result set generated by the subquery:

```
mysql> SELECT customer_id, count(*) num_rentals, sum(amount) tot_payments
    -> FROM payment
    -> GROUP BY customer_id;
+-------------+-------------+--------------+
```

```
| customer_id | num_rentals | tot_payments |
+-------------+-------------+--------------+
|           1 |          32 |       118.68 |
|           2 |          27 |       128.73 |
|           3 |          26 |       135.74 |
|           4 |          22 |        81.78 |
...
|         596 |          28 |        96.72 |
|         597 |          25 |        99.75 |
|         598 |          22 |        83.78 |
|         599 |          19 |        83.81 |
+-------------+-------------+--------------+
599 rows in set (0.03 sec)
```

The subquery is given the name `pymnt` and is joined to the `customer` table via the `customer_id` column. The containing query then retrieves the customer's name from the `customer` table, along with the summary columns from the `pymnt` subquery.

Subqueries used in the `from` clause must be noncorrelated;[1] they are executed first, and the data is held in memory until the containing query finishes execution. Subqueries offer immense flexibility when writing queries, because you can go far beyond the set of available tables to create virtually any view of the data that you desire and then join the results to other tables or subqueries. If you are writing reports or generating data feeds to external systems, you may be able to do things with a single query that used to demand multiple queries or a procedural language to accomplish.

### Data fabrication

Along with using subqueries to summarize existing data, you can use subqueries to generate data that doesn't exist in any form within your database. For example, you may wish to group your customers by the amount of money spent on film rentals, but you want to use group definitions that are not stored in your database. For example, let's say you want to sort your customers into the groups shown in Table 9-1.

Table 9-1. Customer payment groups

| Group name | Lower limit | Upper limit |
|---|---|---|
| Small Fry | 0 | $74.99 |
| Average Joes | $75 | $149.99 |
| Heavy Hitters | $150 | $9,999,999.99 |

To generate these groups within a single query, you will need a way to define these three groups. The first step is to define a query that generates the group definitions:

```
mysql> SELECT 'Small Fry' name, 0 low_limit, 74.99 high_limit
    -> UNION ALL
    -> SELECT 'Average Joes' name, 75 low_limit, 149.99 high_limit
    -> UNION ALL
    -> SELECT 'Heavy Hitters' name, 150 low_limit, 9999999.99 high_limit;
+--------------+-----------+------------+
| name         | low_limit | high_limit |
+--------------+-----------+------------+
| Small Fry    |         0 |      74.99 |
| Average Joes |        75 |     149.99 |
| Heavy Hitters |      150 | 9999999.99 |
+--------------+-----------+------------+
3 rows in set (0.00 sec)
```

I have used the set operator `union all` to merge the results from three separate queries into a single result set. Each query retrieves three literals, and the results from the three queries are put together to generate a result set with three rows and three columns. You now have a query to generate the desired groups, and you can place it into the `from` clause of another query to generate your customer groups:

```
mysql> SELECT pymnt_grps.name, count(*) num_customers
    -> FROM
    ->  (SELECT customer_id,
    ->     count(*) num_rentals, sum(amount) tot_payments
    ->    FROM payment
    ->    GROUP BY customer_id
    ->  ) pymnt
    ->    INNER JOIN
    ->  (SELECT 'Small Fry' name, 0 low_limit, 74.99 high_limit
```

```
    ->   UNION ALL
    ->   SELECT 'Average Joes' name, 75 low_limit, 149.99 high_limit
    ->   UNION ALL
    ->   SELECT 'Heavy Hitters' name, 150 low_limit, 9999999.99 high_limit
    ->  ) pymnt_grps
    ->   ON pymnt.tot_payments
    ->    BETWEEN pymnt_grps.low_limit AND pymnt_grps.high_limit
    -> GROUP BY pymnt_grps.name;
+---------------+---------------+
| name          | num_customers |
+---------------+---------------+
| Average Joes  |           515 |
| Heavy Hitters |            46 |
| Small Fry     |           38 |
+---------------+---------------+
3 rows in set (0.03 sec)
```

The `from` clause contains two subqueries; the first subquery, named `pymnt`, returns the total number of film rentals and total payments for each customer, while the second subquery, named `pymnt_grps`, generates the three customer groupings. The two subqueries are joined by finding which of the three groups each customer belongs to, and the rows are then grouped by the group name in order to count the number of customers in each group.

Of course, you could simply decide to build a permanent (or temporary) table to hold the group definitions instead of using a subquery. Using that approach, you would find your database to be littered with small special-purpose tables after a while, and you wouldn't remember the reason for which most of them were created. Using subqueries, however, you will be able to adhere to a policy where tables are added to a database only when there is a clear business need to store new data.

### Task-oriented subqueries

Let's say that you want to generate a report showing each customer's name, along with their city, the total number of rentals, and the total payment amount. You could accomplish this by joining the `payment`, `customer`, `address`, and `city` tables, and then grouping on the customer's first and last names:

```
mysql> SELECT c.first_name, c.last_name, ct.city,
    ->   sum(p.amount) tot_payments, count(*) tot_rentals
    -> FROM payment p
    ->   INNER JOIN customer c
```

```
    ->   ON p.customer_id = c.customer_id
    ->   INNER JOIN address a
    ->   ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->   ON a.city_id = ct.city_id
    -> GROUP BY c.first_name, c.last_name, ct.city;
+------------+-----------+----------------+-------------+-------------+
| first_name  | last_name  | city           | tot_payments | tot_rentals |
+------------+-----------+----------------+-------------+-------------+
| MARY       | SMITH      | Sasebo         |      118.68 |         32 |
| PATRICIA    | JOHNSON    | San Bernardino  |      128.73 |         27 |
| LINDA      | WILLIAMS   | Athenai        |      135.74 |         26 |
| BARBARA     | JONES      | Myingyan       |       81.78 |         22 |
...
| TERRENCE    | GUNDERSON  | Jinzhou        |      117.70 |         30 |
| ENRIQUE     | FORSYTHE   | Patras         |       96.72 |         28 |
| FREDDIE     | DUGGAN     | Sullana        |       99.75 |         25 |
| WADE       | DELVALLE   | Lausanne       |       83.78 |         22 |
| AUSTIN      | CINTRON    | Tieli          |       83.81 |         19 |
+------------+-----------+----------------+-------------+-------------+
599 rows in set (0.06 sec)
```

This query returns the desired data, but if you look at the query closely, you will see that the `customer`, `address`, and `city` tables are needed only for display purposes and that the `payment` table has everything needed to generate the groupings (`customer_id` and `amount`). Therefore, you could separate out the task of generating the groups into a subquery and then join the other three tables to the table generated by the subquery to achieve the desired end result. Here's the grouping subquery:

```
mysql> SELECT customer_id,
    ->   count(*) tot_rentals, sum(amount) tot_payments
    -> FROM payment
    -> GROUP BY customer_id;
+------------+------------+-------------+
| customer_id | tot_rentals | tot_payments |
+------------+------------+-------------+
|          1 |         32 |      118.68 |
|          2 |         27 |      128.73 |
|          3 |         26 |      135.74 |
|          4 |         22 |       81.78 |
...
|        595 |         30 |      117.70 |
|        596 |         28 |       96.72 |
|        597 |         25 |       99.75 |
```

```
|       598 |        22 |       83.78 |
|       599 |        19 |       83.81 |
+-------------+------------+--------------+
599 rows in set (0.03 sec)
```

This is the heart of the query; the other tables are needed only to provide meaningful strings in place of the `customer_id` value. The next query joins the previous data set to the other three tables:

```
mysql> SELECT c.first_name, c.last_name,
    ->   ct.city,
    ->   pymnt.tot_payments, pymnt.tot_rentals
    -> FROM
    ->  (SELECT customer_id,
    ->    count(*) tot_rentals, sum(amount) tot_payments
    ->   FROM payment
    ->   GROUP BY customer_id
    ->  ) pymnt
    ->   INNER JOIN customer c
    ->   ON pymnt.customer_id = c.customer_id
    ->   INNER JOIN address a
    ->   ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->   ON a.city_id = ct.city_id;
+-------------+------------+----------------+-------------+-------------+
| first_name  | last_name  | city           | tot_payments | tot_rentals |
+-------------+------------+----------------+-------------+-------------+
| MARY        | SMITH      | Sasebo         |       118.68 |         32 |
| PATRICIA    | JOHNSON    | San Bernardino |       128.73 |         27 |
| LINDA       | WILLIAMS   | Athenai        |       135.74 |         26 |
| BARBARA     | JONES      | Myingyan       |        81.78 |         22 |
...
| TERRENCE    | GUNDERSON  | Jinzhou        |       117.70 |         30 |
| ENRIQUE     | FORSYTHE   | Patras         |        96.72 |         28 |
| FREDDIE     | DUGGAN     | Sullana        |        99.75 |         25 |
| WADE        | DELVALLE   | Lausanne       |        83.78 |         22 |
| AUSTIN      | CINTRON    | Tieli          |        83.81 |         19 |
+-------------+------------+----------------+-------------+-------------+
599 rows in set (0.06 sec)
```

I realize that beauty is in the eye of the beholder, but I find this version of the query to be far more satisfying than the big, flat version. This version may execute faster as well,

because the grouping is being done on a single numeric column ( customer_id ) instead of multiple lengthy string columns ( customer.first_name , customer.last_name , city.city ).

### Common table expressions

Common table expressions (a.k.a. CTEs), which are new to MySQL in version 8.0, have been available in other database servers for quite some time. A CTE is a named subquery that appears at the top of a query in a `with clause`, which can contain multiple CTEs separated by commas. Along with making queries more understandable, this feature also allows each CTE to refer to any other CTE defined above it in the same `with` clause. The following example includes three CTEs, where the second refers to the first, and the third refers to the second:

```
mysql> WITH actors_s AS
    ->  (SELECT actor_id, first_name, last_name
    ->   FROM actor
    ->   WHERE last_name LIKE 'S%'
    ->  ),
    ->  actors_s_pg AS
    ->  (SELECT s.actor_id, s.first_name, s.last_name,
    ->    f.film_id, f.title
    ->   FROM actors_s s
    ->    INNER JOIN film_actor fa
    ->    ON s.actor_id = fa.actor_id
    ->    INNER JOIN film f
    ->    ON f.film_id = fa.film_id
    ->   WHERE f.rating = 'PG'
    ->  ),
    ->  actors_s_pg_revenue AS
    ->  (SELECT spg.first_name, spg.last_name, p.amount
    ->   FROM actors_s_pg spg
    ->    INNER JOIN inventory i
    ->    ON i.film_id = spg.film_id
    ->    INNER JOIN rental r
    ->    ON i.inventory_id = r.inventory_id
    ->    INNER JOIN payment p
    ->    ON r.rental_id = p.rental_id
    ->  ) -- end of With clause
    -> SELECT spg_rev.first_name, spg_rev.last_name,
    ->   sum(spg_rev.amount) tot_revenue
    -> FROM actors_s_pg_revenue spg_rev
    -> GROUP BY spg_rev.first_name, spg_rev.last_name
    -> ORDER BY 3 desc;
+------------+-------------+-------------+
```

```
| first_name | last_name   | tot_revenue |
+------------+-------------+-------------+
| NICK       | STALLONE    |      692.21 |
| JEFF       | SILVERSTONE |      652.35 |
| DAN        | STREEP      |      509.02 |
| GROUCHO    | SINATRA     |      457.97 |
| SISSY      | SOBIESKI    |      379.03 |
| JAYNE      | SILVERSTONE |      372.18 |
| CAMERON    | STREEP      |      361.00 |
| JOHN       | SUVARI      |      296.36 |
| JOE        | SWANK       |      177.52 |
+------------+-------------+-------------+
9 rows in set (0.18 sec)
```

This query calculates the total revenues generated from PG-rated film rentals where the cast includes an actor whose last name starts with S. The first subquery ( `actors_s` ) finds all actors whose last name starts with S, the second subquery ( `actors_s_pg` ) joins that data set to the `film` table and filters on films having a PG rating, and the third subquery ( `actors_s_pg_revenue` ) joins that data set to the `payment` table to retrieve the amounts paid to rent any of these films. The final query simply groups the data from `actors_s_pg_revenue` by first/last names and sums the revenues.

---

NOTE

Those who tend to utilize temporary tables to store query results for use in subsequent queries may find CTEs an attractive alternative.

---

## Subqueries as Expression Generators

For this last section of the chapter, I finish where I began: with single-column, single-row scalar subqueries. Along with being used in filter conditions, scalar subqueries may be used wherever an expression can appear, including the `select` and `order by` clauses of a query and the `values` clause of an `insert` statement.

In “Task-oriented subqueries”, I showed you how to use a subquery to separate out the grouping mechanism from the rest of the query. Here’s another version of the same query that uses subqueries for the same purpose, but in a different way:

```
mysql> SELECT
    ->  (SELECT c.first_name FROM customer c
```

```
    ->   WHERE c.customer_id = p.customer_id
    ->  ) first_name,
    ->  (SELECT c.last_name FROM customer c
    ->   WHERE c.customer_id = p.customer_id
    ->  ) last_name,
    ->  (SELECT ct.city
    ->   FROM customer c
    ->   INNER JOIN address a
    ->     ON c.address_id = a.address_id
    ->   INNER JOIN city ct
    ->     ON a.city_id = ct.city_id
    ->   WHERE c.customer_id = p.customer_id
    ->  ) city,
    ->   sum(p.amount) tot_payments,
    ->   count(*) tot_rentals
    -> FROM payment p
    -> GROUP BY p.customer_id;
+------------+-----------+----------------+-------------+-------------+
| first_name | last_name | city           | tot_payments | tot_rentals |
+------------+-----------+----------------+-------------+-------------+
| MARY       | SMITH     | Sasebo         |      118.68 |          32 |
| PATRICIA   | JOHNSON   | San Bernardino |      128.73 |          27 |
| LINDA      | WILLIAMS  | Athenai        |      135.74 |          26 |
| BARBARA    | JONES     | Myingyan       |       81.78 |          22 |
...
| TERRENCE   | GUNDERSON | Jinzhou        |      117.70 |          30 |
| ENRIQUE    | FORSYTHE  | Patras         |       96.72 |          28 |
| FREDDIE    | DUGGAN    | Sullana        |       99.75 |          25 |
| WADE       | DELVALLE  | Lausanne       |       83.78 |          22 |
| AUSTIN     | CINTRON   | Tieli          |       83.81 |          19 |
+------------+-----------+----------------+-------------+-------------+
599 rows in set (0.06 sec)
```

There are two main differences between this query and the earlier version using a subquery in the `from` clause:

- Instead of joining the `customer`, `address`, and `city` tables to the payment data, correlated scalar subqueries are used in the `select` clause to look up the customer's first/last names and city.
- The `customer` table is accessed three times (once in each of the three subqueries) rather than just once.

The `customer` table is accessed three times because scalar subqueries can return only a single column and row, so if we need three columns related to the customer, it is neces-

sary to use three different subqueries.

As previously noted, scalar subqueries can also appear in the `order by` clause. The following query retrieves an actor's first and last names and sorts by the number of films in which the actor appeared:

```
mysql> SELECT a.actor_id, a.first_name, a.last_name
    -> FROM actor a
    -> ORDER BY
    ->  (SELECT count(*) FROM film_actor fa
    ->   WHERE fa.actor_id = a.actor_id) DESC;
+----------+------------+--------------+
| actor_id | first_name  | last_name    |
+----------+------------+--------------+
|      107 | GINA       | DEGENERES    |
|      102 | WALTER     | TORN         |
|      198 | MARY       | KEITEL       |
|      181 | MATTHEW    | CARREY       |
...
|       71 | ADAM       | GRANT        |
|      186 | JULIA      | ZELLWEGER    |
|       35 | JUDY       | DEAN         |
|      199 | JULIA      | FAWCETT      |
|      148 | EMILY      | DEE          |
+----------+------------+--------------+
200 rows in set (0.01 sec)
```

The query uses a correlated scalar subquery in the `order by` clause to return just the number of film appearances, and this value is used solely for sorting purposes.

Along with using correlated scalar subqueries in `select` statements, you can use noncorrelated scalar subqueries to generate values for an `insert` statement. For example, let's say you are going to generate a new row in the `film_actor` table, and you've been given the following data:

- The first and last name of the actor
- The name of the film

You have two choices for how to go about it: execute two queries to retrieve the primary key values from `film` and `actor` and place those values into an `insert` statement or use subqueries to retrieve the two key values from within an `insert` statement. Here's an example of the latter approach:

```
INSERT INTO film_actor (actor_id, film_id, last_update)
VALUES (
 (SELECT actor_id FROM actor
  WHERE first_name = 'JENNIFER' AND last_name = 'DAVIS'),
 (SELECT film_id FROM film
  WHERE title = 'ACE GOLDFINGER'),
 now()
 );
```

Using a single SQL statement, you can create a row in the `film_actor` table and look up two foreign key column values at the same time.

# Subquery Wrap-Up

I covered a lot of ground in this chapter, so it might be a good idea to review it. The examples in this chapter demonstrate subqueries that:

- Return a single column and row, a single column with multiple rows, and multiple columns and rows
- Are independent of the containing statement (noncorrelated subqueries)
- Reference one or more columns from the containing statement (correlated subqueries)
- Are used in conditions that utilize comparison operators as well as the special-purpose operators `in`, `not in`, `exists`, and `not exists`
- Can be found in `select`, `update`, `delete`, and `insert` statements
- Generate result sets that can be joined to other tables (or subqueries) in a query
- Can be used to generate values to populate a table or to populate columns in a query's result set
- Are used in the `select`, `from`, `where`, `having`, and `order by` clauses of queries

Obviously, subqueries are a very versatile tool, so don't feel bad if all these concepts haven't sunk in after reading this chapter for the first time. Keep experimenting with the various uses for subqueries, and you will soon find yourself thinking about how you might utilize a subquery every time you write a nontrivial SQL statement.

# Test Your Knowledge

These exercises are designed to test your understanding of subqueries. Please see Appendix B for the solutions.

## Exercise 9-1

Construct a query against the `film` table that uses a filter condition with a noncorrelated subquery against the `category` table to find all action films ( `category.name = 'Action'` ).

## Exercise 9-2

Rework the query from Exercise 9-1 using a *correlated* subquery against the `category` and `film_category` tables to achieve the same results.

## Exercise 9-3

Join the following query to a subquery against the `film_actor` table to show the level of each actor:

```
SELECT 'Hollywood Star' level, 30 min_roles, 99999 max_roles
UNION ALL
SELECT 'Prolific Actor' level, 20 min_roles, 29 max_roles
UNION ALL
SELECT 'Newcomer' level, 1 min_roles, 19 max_roles
```

The subquery against the `film_actor` table should count the number of rows for each actor using `group by actor_id` , and the count should be compared to the `min_roles` / `max_roles` columns to determine which level each actor belongs to.

1 Actually, depending on which database server you are using, you might be able to include correlated subqueries in your `from` clause by using `cross apply` or `outer apply` , but these features are beyond the scope of this book.

Previous chapter

< 8. Grouping and Aggregates

Next chapter

10. Joins Revisited >